

TradeScope AI — Complete Development Plan

1. Project Vision

TradeScope AI ek complete AI-powered trading platform hoga jahan:

- Client multiple trading accounts manage kar sakega.
- Live market prices, charts aur trading terminal use kar sakega.
- Manual aur AI-generated trades execute ho saken gi.
- AI risk settings client level par configurable honghi.
- Anonymous live traders leaderboard available hoga.
- Deposits, withdrawals aur transaction history manage hogi.
- Admin poore platform, users, funds, risk aur AI engine ko control kar sakega.

Platform ko do operating modes support karne chahiye:

Demo / Simulation Mode

- Simulated market prices ya external market-feed prices.
- Simulated trade execution.
- Virtual deposits and withdrawals.
- AI performance testing.
- Demo clients ke liye safe environment.

Live Broker Mode

- Licensed market-data provider.
- Broker execution API.
- Real deposit/payment integrations.
- KYC/AML checks.
- Real account balances and settlement.
- Jurisdiction-specific regulatory compliance.

System ko is tarah build karna chahiye ke demo se live mode par jane ke liye frontend rewrite na karna pare. Sirf provider adapters replace hon.

2. Recommended Technology Architecture

TradeScope AI ke current agreed stack ke mutabiq:

Frontend

- Expo SDK 56
- React Native
- React Native Web
- TypeScript
- Expo Router

- TanStack Query for API state
- Zustand for local trading state
- WebSocket client for live updates
- Trading chart library with candlestick, crosshair, zoom and pan support
- Cloudflare Pages deployment

Ek hi frontend bundle mein route-based separation:

```
/client/*
/admin/*
```

Backend

- FastAPI
- Python 3.12
- Pydantic validation
- SQLAlchemy or Supabase Python client
- REST APIs for commands and standard queries
- WebSockets for live data
- Background workers for AI, reports and settlement
- Render deployment

Database and Storage

- Supabase PostgreSQL
- Supabase Auth
- Supabase Storage
- Row Level Security
- Realtime database events where suitable
- Redis recommended for production caching, distributed locks and queues

External Integrations

- Market-data provider
- Broker/execution provider
- Payment gateways
- Bank-transfer processing
- Crypto-payment provider
- News-feed providers
- Email/SMS/push notification provider
- LiveKit for web calling where required

3. High-Level System Architecture

```
flowchart TB
    subgraph Frontend
        C[Client Zone]
```

```

    A[Admin Console]
end

subgraph API["FastAPI Application"]
    AUTH[Authentication and RBAC]
    USER[User and Account Service]
    MARKET[Market Data Service]
    TRADE[Trading and Order Service]
    POSITION[Position and Portfolio Service]
    AI[AI Trading Engine]
    RISK[Risk Management Engine]
    LEDGER[Ledger and Finance Service]
    PAYMENT[Deposit and Withdrawal Service]
    LIVE[Live Traders Service]
    NEWS[News Service]
    NOTIFY[Notification Service]
    ADMIN[Admin Control Service]
    AUDIT[Audit Service]
    RT[Realtime Event Gateway]
end

subgraph Data
    DB[(Supabase PostgreSQL)]
    STORAGE[(Supabase Storage)]
    CACHE[(Redis / Queue)]
end

subgraph External
    MD[Market Data Provider]
    BROKER[Broker Execution API]
    PSP[Payment Providers]
    NEWSP[Licensed News Providers]
    MSG[Email / SMS / Push]
end

C --> AUTH
A --> AUTH

C --> USER
C --> MARKET
C --> TRADE
C --> POSITION
C --> AI
C --> PAYMENT
C --> LIVE
C --> NEWS

A --> ADMIN
A --> LEDGER
A --> RISK
A --> AI

```

MARKET <--> MD
TRADE <--> BROKER
PAYMENT <--> PSP
NEWS <--> NEWSP
NOTIFY <--> MSG

USER --> DB
TRADE --> DB
POSITION --> DB
AI --> DB
RISK --> DB
LEDGER --> DB
PAYMENT --> DB
LIVE --> DB
ADMIN --> DB
AUDIT --> DB

AI <--> CACHE
MARKET <--> CACHE
RT <--> CACHE

DB --> RT
RT --> C
RT --> A

USER --> STORAGE

Architecture Approach

Initial implementation ko **modular monolith** rakha jaye:

- Single FastAPI application.
- Har business domain ka separate module.
- Shared database.
- Internal service interfaces.
- Event-driven communication.

Future mein high-load services ko independently split kiya ja sakta hai:

- Market data service
- AI engine
- WebSocket gateway
- Trade execution service
- Notification service

4. User Roles and Access Control

Client Role

Client sirf:

- Apna profile
- Apne accounts
- Apne trades
- Apne transactions
- Apni AI settings
- Public anonymous leaderboard

access kar sakega.

Admin Roles

Super Admin

Poore platform ka control.

User Administrator

- User create/update
- Account activate/suspend
- AI enable/disable
- User status management

Finance Administrator

- Deposits review
- Withdrawals approve/reject
- Balance adjustments
- Ledger access
- Financial exports

Risk Manager

- User risk limits
- Global trade limits
- Emergency stop
- Margin and exposure monitoring

Compliance Administrator

- KYC review
- Identity checks
- Suspicious activity review
- Account restrictions

Support Agent

- Limited user information
- Account support
- No balance adjustment access
- No emergency controls

Auditor

- Read-only access
- Audit logs
- Reports
- Transaction history

Every admin permission server-side validate honi chahiye. Frontend par button hide karna security control nahi samjha jayega.

5. Client-Side Feature Specifications

5.1 Authentication and Client Foundation

Although overview mein authentication separately mention nahi hai, platform ke liye yeh foundation required hai.

Features

- Registration
- Login
- Logout
- Email verification
- Forgot password
- Reset password
- Optional two-factor authentication
- Session management
- Device/session history
- Profile management
- KYC status
- Terms and risk-disclosure acceptance

Required Data

- Full name
- Email
- Phone
- Country
- Preferred currency
- Timezone
- Account status
- KYC status
- Risk-disclosure version

- Terms version

Account Statuses

```
pending_verification
active
restricted
suspended
closed
```

Security Requirements

- Password Supabase Auth handle kare.
- Access JWT server-side verify ho.
- Refresh-token rotation.
- Brute-force rate limiting.
- Registration endpoint rate limiting.
- Suspicious login notifications.
- Sensitive changes ke liye recent authentication.

5.2 AI Trading Dashboard

Dashboard client ka primary overview hoga.

Portfolio Summary

Dashboard cards:

- Total balance
- Cash
- Equity
- Buying power
- Used margin
- Free margin
- Margin level
- Daily realized P/L
- Daily unrealized P/L
- Total open positions

Balance Definitions

Balance

Closed trades aur settled transactions ke baad available account value.

Equity

$$\text{Equity} = \text{Balance} + \text{Unrealized P/L}$$

Free Margin

$$\text{Free Margin} = \text{Equity} - \text{Used Margin}$$

Margin Level

$$\text{Margin Level \%} = \text{Equity} / \text{Used Margin} \times 100$$

Used margin zero hone par margin level ko null ya unlimited display kiya ja sakta hai.

Buying Power

Buying power instrument, leverage aur account rules ke mutabiq calculate hogi.

Simplified representation:

$$\text{Buying Power} = \text{Free Margin} \times \text{Allowed Leverage}$$

Actual calculation broker aur instrument rules ke mutabiq honi chahiye.

Portfolio Chart

Chart options:

- Today
- 7 days
- 30 days
- 90 days
- All time

Chart metrics:

- Equity curve
- Balance curve
- Realized P/L
- Unrealized P/L
- Drawdown

Portfolio snapshots fixed interval par save hon:

- Every 1–5 minutes for active accounts
- On every fill

- On deposit/withdrawal
- On position close

AI Market Analysis

Each analysis card mein:

- Symbol
- Direction: Buy, Sell or Neutral
- Confidence score
- Current market price
- Suggested entry
- Stop loss
- Take profit
- Risk/reward ratio
- Strategy name
- Analysis timestamp
- Expiry timestamp
- AI reasoning summary
- Risk warnings

Confidence score ko guaranteed probability ke taur par present nahi karna chahiye. Score calibrated model confidence hoga.

Open Trades Table

Columns:

- Symbol
- Direction
- Quantity
- Entry price
- Current price
- Stop loss
- Take profit
- Unrealized P/L
- P/L percentage
- Open time
- AI/manual source
- Close action

Live updates:

```
market.tick
position.updated
account.metrics.updated
```

Daily Goal Progress

Show:

- Daily profit target
- Current daily realized P/L
- Unrealized P/L
- Target progress percentage
- Daily loss limit
- Remaining loss capacity
- Trades used
- Remaining trades
- Risk status

Risk statuses:

```
safe
warning
near_limit
blocked
```

Live News Feed

News item fields:

- Headline
- Provider
- Published time
- Related symbols
- Sentiment
- Impact level
- Source link

Reuters, Bloomberg aur CNBC content ke liye licensed feeds, approved APIs ya headline-link integrations required hongi. Full copyrighted articles ko unauthorized scrape aur republish nahi karna chahiye.

AI Order Board

Each proposed order:

- Symbol
- Buy/sell
- Entry range
- Current price
- Stop loss
- Take profit
- Position size
- Expected risk

- Risk/reward ratio
- Confidence score
- Strategy
- Valid until
- Approve/reject action where manual confirmation mode enabled ho

Order-board states:

```
proposed
risk_reviewed
approved
rejected
expired
executed
```

5.3 Web Trader

Web Trader ko TradingView/MT5-style experience dena hoga.

Symbol Search

Search by:

- Symbol code
- Company name
- Asset name
- Asset category
- Exchange

Asset categories:

- Forex
- Stocks
- Indices
- Commodities
- Crypto
- ETFs, where supported

Quote Panel

Display:

- Symbol
- Bid
- Ask
- Spread
- Daily change
- Daily change percentage

- Market status
- High
- Low
- Volume
- Last update time

Chart Requirements

- Candlestick chart
- Real OHLC candle proportions
- Live candle movement
- Historical candles
- Horizontal zoom
- Vertical price zoom
- Mouse-wheel zoom
- Drag/pan
- Crosshair
- Time and price tooltip
- Auto-scale
- Reset chart
- Full-screen mode
- Responsive layout
- Current-price line
- Bid/ask lines
- Entry, stop-loss and take-profit lines
- Open-position markers
- Executed-order markers

Candle Update Logic

Incoming tick par:

1. Current timeframe bucket identify hoga.
2. Existing candle ka close update hoga.
3. Price high se zyada ho to high update hoga.
4. Price low se kam ho to low update hoga.
5. Volume update hoga.
6. Timeframe complete hone par new candle create hogi.

Example:

```
open = first tick price
high = highest tick price
low = lowest tick price
close = latest tick price
volume = accumulated tick volume
```

Frontend ko har tick par complete candle history reload nahi karni chahiye. Sirf current candle update honi chahiye.

Timeframes

Recommended:

1m
5m
15m
30m
1h
4h
1d
1w

Order Ticket

Order types:

- Market
- Limit
- Stop
- Stop-limit, where supported
- Stop loss
- Take profit
- Trailing stop, later phase

Order fields:

- Account
- Symbol
- Direction
- Order type
- Quantity
- Entry price
- Stop loss
- Take profit
- Time in force
- Estimated margin
- Estimated commission
- Estimated spread cost
- Risk percentage

One-Click Trading

One-click trading se pehle client ko explicit consent dena hoga.

Controls:

- Default quantity
- Buy button

- Sell button
- Confirmation setting
- Max position-size validation
- Available margin validation
- Market-open validation

Position Management

Client actions:

- Close full position
- Close partial position
- Modify stop loss
- Modify take profit
- Cancel pending order
- Reverse position, optional
- Close all positions, optional

Order State Machine

```
draft
→ validating
→ accepted
→ submitted
→ partially_filled
→ filled
```

Alternative endings:

```
rejected
canceled
expired
failed
```

Every state change `order_events` table mein immutable record ke taur par save hoga.

5.4 Live Traders

Live Traders section anonymous social-proof aur engagement module hoga.

Public Trader Identity

Real names expose nahi honge.

Example:

TRD-8K21
TRD-A94P
TRD-X72M

Public profile data:

- Trader public ID
- AI risk profile
- Win rate
- Realized profit
- Daily profit
- Total trades
- Average trade duration
- Current winning streak
- Rank

Leaderboard Filters

- Today
- This week
- This month
- All time
- Conservative
- Moderate
- Aggressive

Leaderboard Ranking

Ranking configurable formula se calculate ho:

```
score =  
weighted_return  
+ consistency_score  
+ win_rate_score  
- drawdown_penalty  
- risk_penalty
```

Sirf profit par ranking karna risky behavior encourage kar sakta hai. Drawdown aur consistency ko include karna chahiye.

Robot Activity Feed

Example events:

```
TRD-8K21 AI bot opened BUY EUR/USD
TRD-X72M closed BTC/USD with +$145.20
TRD-A94P moved stop loss to break-even
```

Event data:

- Anonymous trader ID
- Symbol
- Action
- Direction
- Profit, if closed
- Timestamp
- AI strategy category

Privacy Rules

- Feature user opt-in ho.
- Real name, email, account number ya balance expose na ho.
- Position size optional masked ho.
- Large trades ko delayed display kiya ja sakta hai.
- Suspended users leaderboard se remove hon.
- Test/admin accounts exclude hon.

Realtime Updates

WebSocket events:

```
leaderboard.updated
robot_activity.created
trader_rank.changed
```

5.5 Account Management

Multiple Trading Accounts

Ek user ke multiple accounts ho sakte hain.

Example:

```
Main Trading Account
AI Conservative Account
AI Aggressive Account
Demo Account
```

Account fields:

- Account number
- Account name
- Account type
- Base currency
- Leverage
- Balance
- Equity
- Buying power
- Status
- AI enabled
- Demo/live mode

Account statuses:

```
pending
active
restricted
suspended
closed
```

Account Switcher

Account switch hote hi update hon:

- Dashboard metrics
- Positions
- Orders
- Transactions
- AI settings
- Risk limits
- WebSocket subscriptions

Frontend ko selected account ID request headers ya route context mein pass karna hoga. Backend ownership verify karega.

Deposits

Deposit methods:

- Bank transfer
- Credit/debit card
- Crypto

Deposit statuses:

```
created
awaiting_payment
```

```
processing
pending_review
completed
failed
canceled
refunded
```

Deposit flow:

1. Client method select kare.
2. Backend deposit intent create kare.
3. Payment provider session create ho.
4. Provider webhook receive ho.
5. Webhook signature verify ho.
6. Duplicate webhook idempotently ignore ho.
7. Ledger journal create ho.
8. Account balance update ho.
9. Client ko realtime notification mile.

Withdrawals

Withdrawal destination:

- Saved bank account
- Saved card, where supported
- Crypto-wallet address

Withdrawal statuses:

```
requested
pending_review
approved
processing
completed
rejected
canceled
failed
```

Validations:

- Sufficient withdrawable balance
- Open-position margin requirement
- Daily withdrawal limit
- KYC status
- Destination ownership
- Account restrictions
- Duplicate request detection
- Compliance review

Transaction History

Transaction types:

- Deposit
- Withdrawal
- Trade settlement
- Commission
- Spread charge
- Financing/swap
- Admin adjustment
- Refund
- Bonus, if applicable

Filters:

- Account
- Date
- Type
- Status
- Amount
- Reference

Export:

- CSV
 - PDF statement, later phase
-

5.6 AI Settings

Risk Profiles

Conservative

- Smaller position sizes
- Lower trade frequency
- Tighter daily loss limit
- Higher confidence threshold
- Lower simultaneous exposure

Moderate

- Balanced risk and opportunity
- Medium trade frequency
- Medium position size

Aggressive

- Larger position sizes
- More simultaneous trades

- Higher drawdown tolerance
- Lower minimum confidence threshold, within global limits

User-Configurable Fields

- Risk profile
- Maximum daily trades
- Maximum open positions
- Maximum position size
- Maximum position size percentage
- Daily loss limit
- Daily profit target
- Allowed instruments
- Allowed trading sessions
- AI auto-trading enabled
- Manual approval required
- Stop after profit target
- Stop after loss limit

Limit Hierarchy

Effective AI setting calculate hogi:

```
Platform Global Limit
    ↓
Account-Type Limit
    ↓
Admin User Override
    ↓
Client Selected Setting
```

Client kabhi admin/global maximum se zyada permissive value set nahi kar sakega.

Example:

```
Platform max daily trades = 20
Admin override = 10
Client selection = 15
Effective limit = 10
```

AI Disable Conditions

AI automatically disable ya pause ho:

- Daily loss limit reached
- Margin below threshold
- Emergency stop active
- Account suspended

- Market feed unavailable
 - Broker unavailable
 - Repeated order rejection
 - Abnormal volatility
 - AI session error
 - User manually pauses AI
-

6. Admin Console Feature Specifications

6.1 Admin Authentication and RBAC

Features:

- Secure admin login
- Mandatory two-factor authentication
- Role-based permissions
- Session expiration
- Device history
- IP and login audit
- Sensitive action re-authentication

Sensitive actions:

- Balance adjustment
 - Withdrawal approval
 - User suspension
 - AI emergency stop
 - Global settings change
 - Role assignment
-

6.2 User Management

User Directory

Columns:

- User
- Email
- Public/client ID
- Country
- KYC status
- Account status
- Trading accounts
- Total balance
- Total equity
- AI status
- Registration date

- Last login

Filters:

- Active
- Pending
- Restricted
- Suspended
- KYC pending
- AI enabled
- High-risk user

User Detail

Tabs:

- Profile
- KYC
- Trading accounts
- Positions
- Orders
- Transactions
- AI settings
- Risk limits
- Login sessions
- Admin notes
- Audit activity

Admin Actions

- Create user
- Update profile
- Verify email, controlled
- Activate
- Restrict
- Suspend
- Reactivate
- Close account
- Enable AI
- Disable AI
- Reset security sessions
- Add internal note

Every action ke liye:

- Reason required
- Admin identity stored
- Previous value stored
- New value stored
- Timestamp stored

6.3 Balance Control

Admin ko balance directly overwrite nahi karna chahiye.

Balance changes hamesha ledger adjustment ke through hon.

Adjustment Types

- Credit correction
- Debit correction
- Promotional credit
- Fee reversal
- Trade correction
- Refund
- Manual settlement

Required fields:

- User
- Trading account
- Amount
- Currency
- Adjustment type
- Reason
- Supporting evidence
- Approval reference

Large adjustments ke liye maker-checker approval recommended hai:

Finance Admin creates adjustment

↓

Second authorized admin approves

↓

Ledger journal posts

Platform Metrics

- Total assets under management
- Total client balance
- Total client equity
- Total open exposure
- Used margin
- Free margin
- Pending deposits
- Pending withdrawals
- Daily deposits
- Daily withdrawals
- Daily realized P/L
- Platform commission revenue

6.4 Transaction Oversight

Admin can:

- View all transactions
- Filter by type
- Filter by amount
- Filter by user
- Filter by provider
- Filter by date
- Filter by status
- View ledger entries
- View webhook history
- Export reports

Transaction detail:

- Transaction ID
- External reference
- User
- Account
- Amount
- Currency
- Fee
- Net amount
- Method
- Provider
- Status history
- Ledger journal
- Risk flags
- Admin notes

6.5 Withdrawal Management

Withdrawal queue:

- Pending review
- Compliance review
- Approved
- Processing
- Failed

Admin checks:

- KYC complete
- Sufficient balance
- Destination verified
- Recent deposit source

- Open margin exposure
- Suspicious pattern
- Withdrawal velocity
- Previous failed attempts

Actions:

- Approve
- Reject
- Request additional information
- Hold
- Retry provider payout
- Mark completed, restricted permission

Reject action ke liye reason mandatory hoga.

6.6 AI Control Panel

Global AI Parameters

- Global AI enabled
- Minimum confidence threshold
- Maximum daily trades
- Maximum position size
- Maximum platform exposure
- Maximum symbol exposure
- Maximum user exposure
- Default stop loss
- Default take profit
- Maximum drawdown
- Maximum open positions
- Allowed instruments
- Allowed market sessions

Per-User Overrides

- AI enable/disable
- Risk profile
- Position-size cap
- Daily trade cap
- Loss limit
- Profit target
- Instrument restrictions
- Strategy restrictions

AI Monitoring

Dashboard:

- Active AI sessions
- Orders proposed
- Orders approved
- Orders rejected by risk
- Orders executed
- Daily win rate
- Realized AI P/L
- AI drawdown
- Provider errors
- Strategy performance
- Confidence distribution

Emergency Stop

Emergency actions should be separate:

Pause New AI Trades

- New AI signals stop.
- Existing positions remain open.
- Pending AI orders cancel based on configuration.

Cancel Pending AI Orders

- All pending AI-generated orders cancel.

Close AI Positions

- Existing AI positions market close.
- High-impact action.
- Two-person approval recommended.

Global Trading Halt

- Manual and AI trading both halt.
- Only risk-reducing close actions allowed.

Emergency-stop event mein:

- Admin
- Reason
- Scope
- Timestamp
- Affected accounts
- Result
- Failed actions

record hon.

6.7 Platform Settings

Settings categories:

Trading

- Default leverage
- Allowed leverage ranges
- Minimum order size
- Maximum order size
- Maximum open positions
- Supported instruments
- Trading-hours rules

Deposits

- Minimum deposit
- Maximum deposit
- Supported currencies
- Payment methods
- Deposit fees

Withdrawals

- Minimum withdrawal
- Maximum withdrawal
- Daily limit
- Withdrawal fee
- Review threshold

Fees

- Commission
- Spread markup
- Financing/swap fee
- Account fee
- Inactivity fee, if applicable

Risk

- Margin-call level
- Stop-out level
- Platform exposure limits
- Symbol exposure limits
- Volatility controls

Application

- Maintenance mode
- Registration enabled
- Default base currency

- Default timezone
- Support contact
- Terms version
- Privacy-policy version

All settings versioned hon. Historical version aur change history available ho.

7. AI Trading Engine Design

AI Processing Pipeline

```

flowchart LR
    MD[Market Data] --> NORMALIZE[Normalize and Validate]
    NORMALIZE --> FEATURES[Technical and Market Features]
    FEATURES --> STRATEGY[Strategy Engine]
    STRATEGY --> SCORE[Signal and Confidence Score]
    SCORE --> RISK[Risk Validation]
    RISK -->|Rejected| REJECT[Store Rejection Reason]
    RISK -->|Approved| PROPOSAL[Create Order Proposal]
    PROPOSAL --> MODE{Trading Mode}
    MODE -->|Manual Approval| BOARD[AI Order Board]
    MODE -->|Auto Trading| ORDER[Submit Order]
    BOARD -->|Client Approves| ORDER
    ORDER --> EXECUTE[Execution Adapter]
    EXECUTE --> FILL[Fill Processing]
    FILL --> POSITION[Position and Ledger Update]
    POSITION --> LEARN[Performance Analytics]
  
```

AI Inputs

- OHLC candles
- Live ticks
- Volume
- Spread
- Volatility
- Technical indicators
- Market-session information
- Existing account exposure
- Open positions
- Daily P/L
- News sentiment, where licensed
- Platform risk conditions

AI Output

```

{
  "symbol": "EURUSD",

```

```

"direction": "BUY",
"confidence": 0.82,
"entry_price": 1.0852,
"stop_loss": 1.0810,
"take_profit": 1.0930,
"risk_reward_ratio": 1.86,
"strategy_id": "trend-following-v2",
"valid_until": "timestamp",
"reason_codes": [
  "trend_alignment",
  "volume_confirmation",
  "acceptable_spread"
]
}

```

AI Decision Record

Every AI decision save honi chahiye, chahe order execute ho ya reject.

Stored information:

- Model version
- Strategy version
- Input snapshot reference
- Confidence
- Suggested order
- Risk checks
- Rejection reason
- Execution result
- Final P/L

Is se AI performance audit aur strategy comparison possible hoga.

Risk Validation Sequence

1. Global emergency-stop check
2. User/account status check
3. AI enabled check
4. Market status check
5. Feed freshness check
6. Daily loss check
7. Daily trade-count check
8. Position-count check
9. Position-size check
10. Symbol exposure check
11. Total exposure check
12. Margin check
13. Spread check
14. Volatility check
15. Duplicate-signal check

8. Trading Execution Architecture

Execution provider abstraction required hogi.

```
class ExecutionAdapter:
    async def place_order(self, order): ...
    async def cancel_order(self, provider_order_id): ...
    async def modify_order(self, provider_order_id, changes): ...
    async def get_order_status(self, provider_order_id): ...
    async def get_positions(self, account_id): ...
```

Implementations:

```
SimulationExecutionAdapter
BrokerExecutionAdapter
```

Market-data abstraction:

```
SimulationMarketDataProvider
LiveMarketDataProvider
```

Payment abstraction:

```
ManualBankTransferProvider
CardPaymentProvider
CryptoPaymentProvider
```

Is approach se business logic provider-specific code se separate rahegi.

9. P/L and Position Calculations

Long Position

```
Unrealized P/L =
Current Bid - Entry Price
× Quantity
× Contract Multiplier
```

Short Position

```
Unrealized P/L =  
Entry Price - Current Ask  
× Quantity  
× Contract Multiplier
```

Fees and conversion separately apply hon.

Realized P/L

```
Realized P/L =  
Exit Value  
- Entry Value  
- Commission  
- Financing  
- Other Fees
```

Position Aggregation

Same symbol ke multiple fills ko:

- Net-position model
- Hedging model

mein manage kiya ja sakta hai.

Account type mein define ho:

```
netting  
hedging
```

10. Financial Ledger Design

Financial values ke liye editable balance field primary source nahi hona chahiye.

Double-entry ledger source of truth hoga.

Example deposit:

```
Debit: Platform Cash / Clearing Account  
Credit: Client Liability Account
```

Example withdrawal:

Debit: Client Liability Account
Credit: Platform Cash / Clearing Account

Every journal:

- Balanced ho
- Total debit = total credit
- Immutable ho
- Reference transaction se linked ho
- Reversal new journal ke through ho

Posted ledger entry ko edit/delete nahi kiya jayega.

11. Complete Logical ERD

erDiagram

```
AUTH_USERS {
    uuid id PK
    string email
    datetime created_at
    datetime last_sign_in_at
}
```

```
USER_PROFILES {
    uuid id PK
    uuid auth_user_id FK
    string client_code
    string full_name
    string phone
    string country
    string base_currency
    string status
    string kyc_status
    datetime created_at
}
```

```
KYC_PROFILES {
    uuid id PK
    uuid user_id FK
    string status
    string risk_level
    datetime submitted_at
    datetime reviewed_at
    uuid reviewed_by FK
}
```

```

}

KYC_DOCUMENTS {
    uuid id PK
    uuid kyc_profile_id FK
    string document_type
    string storage_path
    string status
    datetime created_at
}

ROLES {
    uuid id PK
    string code
    string name
}

PERMISSIONS {
    uuid id PK
    string code
    string name
}

USER_ROLES {
    uuid id PK
    uuid user_id FK
    uuid role_id FK
}

ROLE_PERMISSIONS {
    uuid id PK
    uuid role_id FK
    uuid permission_id FK
}

TRADING_ACCOUNTS {
    uuid id PK
    uuid user_id FK
    string account_number
    string account_name
    string account_type
    string environment
    string base_currency
    decimal leverage
    string position_mode
    string status
    boolean ai_enabled
    datetime created_at
}

ACCOUNT_SETTINGS {

```

```

    uuid id PK
    uuid account_id FK
    boolean one_click_enabled
    decimal default_order_size
    boolean manual_ai_approval
    json preferences
    datetime updated_at
}

ACCOUNT_RISK_LIMITS {
    uuid id PK
    uuid account_id FK
    string risk_profile
    int max_daily_trades
    int max_open_positions
    decimal max_position_size
    decimal daily_loss_limit
    decimal daily_profit_target
    decimal max_drawdown
    json allowed_instruments
    datetime updated_at
}

INSTRUMENTS {
    uuid id PK
    string symbol
    string name
    string asset_class
    string exchange
    string quote_currency
    decimal contract_size
    decimal tick_size
    decimal tick_value
    string status
}

MARKET_QUOTES {
    uuid id PK
    uuid instrument_id FK
    decimal bid
    decimal ask
    decimal last
    decimal volume
    datetime quoted_at
}

CANDLES {
    uuid id PK
    uuid instrument_id FK
    string timeframe
    datetime open_time

```

```

        decimal open
        decimal high
        decimal low
        decimal close
        decimal volume
    }

    ORDERS {
        uuid id PK
        uuid account_id FK
        uuid instrument_id FK
        uuid ai_signal_id FK
        string client_order_id
        string provider_order_id
        string source
        string side
        string order_type
        decimal quantity
        decimal requested_price
        decimal stop_loss
        decimal take_profit
        string time_in_force
        string status
        datetime created_at
    }

    ORDER_EVENTS {
        uuid id PK
        uuid order_id FK
        string previous_status
        string new_status
        string reason
        json provider_payload
        datetime created_at
    }

    FILLS {
        uuid id PK
        uuid order_id FK
        string provider_fill_id
        decimal quantity
        decimal price
        decimal commission
        datetime filled_at
    }

    POSITIONS {
        uuid id PK
        uuid account_id FK
        uuid instrument_id FK
        string side

```

```

    decimal quantity
    decimal average_entry_price
    decimal current_price
    decimal stop_loss
    decimal take_profit
    decimal unrealized_pnl
    decimal realized_pnl
    string status
    datetime opened_at
    datetime closed_at
}

POSITION_EVENTS {
    uuid id PK
    uuid position_id FK
    string event_type
    decimal quantity
    decimal price
    json metadata
    datetime created_at
}

AI_PROFILES {
    uuid id PK
    uuid account_id FK
    string risk_profile
    boolean enabled
    boolean auto_trade
    decimal minimum_confidence
    json settings
    datetime updated_at
}

AI_STRATEGIES {
    uuid id PK
    string code
    string name
    string version
    string status
    json configuration
}

AI_SESSIONS {
    uuid id PK
    uuid account_id FK
    string status
    datetime started_at
    datetime stopped_at
    string stop_reason
}

```

```

AI_SIGNALS {
  uuid id PK
  uuid account_id FK
  uuid instrument_id FK
  uuid strategy_id FK
  string direction
  decimal confidence
  decimal entry_price
  decimal stop_loss
  decimal take_profit
  decimal suggested_quantity
  string status
  datetime valid_until
  datetime created_at
}

```

```

AI_DECISIONS {
  uuid id PK
  uuid signal_id FK
  uuid session_id FK
  string model_version
  json input_snapshot
  json risk_checks
  string decision
  string reason
  datetime created_at
}

```

```

PAYMENT_METHODS {
  uuid id PK
  uuid user_id FK
  string type
  string provider
  string masked_identifier
  boolean verified
  json provider_metadata
  datetime created_at
}

```

```

DEPOSITS {
  uuid id PK
  uuid account_id FK
  uuid payment_method_id FK
  decimal amount
  string currency
  decimal fee
  decimal net_amount
  string provider_reference
  string status
  datetime created_at
  datetime completed_at
}

```

```

}

WITHDRAWALS {
    uuid id PK
    uuid account_id FK
    uuid payment_method_id FK
    decimal amount
    string currency
    decimal fee
    decimal net_amount
    string provider_reference
    string status
    uuid reviewed_by FK
    string review_reason
    datetime created_at
    datetime completed_at
}

TRANSACTIONS {
    uuid id PK
    uuid account_id FK
    uuid deposit_id FK
    uuid withdrawal_id FK
    string transaction_type
    decimal amount
    string currency
    string status
    string reference
    datetime created_at
}

LEDGER_ACCOUNTS {
    uuid id PK
    uuid trading_account_id FK
    string account_code
    string account_type
    string currency
    string status
}

LEDGER_JOURNALS {
    uuid id PK
    string reference_type
    uuid reference_id
    string description
    string status
    datetime posted_at
    uuid created_by FK
}

LEDGER_ENTRIES {

```

```

    uuid id PK
    uuid journal_id FK
    uuid ledger_account_id FK
    string entry_type
    decimal amount
    string currency
    datetime created_at
}

ADMIN_ADJUSTMENTS {
    uuid id PK
    uuid account_id FK
    uuid journal_id FK
    decimal amount
    string adjustment_type
    string reason
    uuid created_by FK
    uuid approved_by FK
    string status
    datetime created_at
}

PORTFOLIO_SNAPSHOTS {
    uuid id PK
    uuid account_id FK
    decimal balance
    decimal equity
    decimal buying_power
    decimal used_margin
    decimal free_margin
    decimal realized_pnl
    decimal unrealized_pnl
    datetime captured_at
}

DAILY_ACCOUNT_METRICS {
    uuid id PK
    uuid account_id FK
    date metric_date
    int trades_count
    int winning_trades
    int losing_trades
    decimal realized_pnl
    decimal max_drawdown
    decimal commission
}

PUBLIC_TRADER_PROFILES {
    uuid id PK
    uuid account_id FK
    string public_trader_code

```

```

        boolean leaderboard_enabled
        string public_risk_profile
        datetime created_at
    }

    LEADERBOARD_SNAPSHOTS {
        uuid id PK
        uuid public_trader_id FK
        string period
        int rank
        decimal score
        decimal profit
        decimal return_percentage
        decimal win_rate
        decimal drawdown
        datetime captured_at
    }

    ROBOT_ACTIVITY_EVENTS {
        uuid id PK
        uuid public_trader_id FK
        uuid instrument_id FK
        string activity_type
        string direction
        decimal public_profit
        datetime occurred_at
    }

    NEWS_ITEMS {
        uuid id PK
        string provider
        string external_id
        string headline
        string source_url
        string sentiment
        string impact_level
        json related_symbols
        datetime published_at
    }

    NOTIFICATIONS {
        uuid id PK
        uuid user_id FK
        string type
        string title
        string message
        json payload
        boolean read
        datetime created_at
    }

```

```

PLATFORM_SETTINGS {
    uuid id PK
    string category
    string setting_key
    json setting_value
    int version
    boolean active
    uuid updated_by FK
    datetime updated_at
}

USER_OVERRIDES {
    uuid id PK
    uuid account_id FK
    string override_type
    json override_value
    string reason
    uuid created_by FK
    datetime expires_at
}

EMERGENCY_CONTROLS {
    uuid id PK
    string scope
    string action
    string status
    string reason
    uuid triggered_by FK
    datetime triggered_at
    datetime cleared_at
    uuid cleared_by FK
}

RISK_EVENTS {
    uuid id PK
    uuid account_id FK
    uuid order_id FK
    string event_type
    string severity
    string reason
    json details
    datetime created_at
}

AUDIT_LOGS {
    uuid id PK
    uuid actor_user_id FK
    string actor_role
    string action
    string resource_type
    uuid resource_id
}

```

```

    json previous_value
    json new_value
    string ip_address
    datetime created_at
}

WEBHOOK_EVENTS {
    uuid id PK
    string provider
    string external_event_id
    string event_type
    json payload
    string processing_status
    int attempt_count
    datetime received_at
    datetime processed_at
}

IDEMPOTENCY_KEYS {
    uuid id PK
    uuid user_id FK
    string idempotency_key
    string request_hash
    string resource_type
    uuid resource_id
    datetime expires_at
}

AUTH_USERS ||--|| USER_PROFILES : owns
USER_PROFILES ||--o| KYC_PROFILES : has
KYC_PROFILES ||--o{ KYC_DOCUMENTS : contains

USER_PROFILES ||--o{ USER_ROLES : assigned
ROLES ||--o{ USER_ROLES : maps
ROLES ||--o{ ROLE_PERMISSIONS : contains
PERMISSIONS ||--o{ ROLE_PERMISSIONS : maps

USER_PROFILES ||--o{ TRADING_ACCOUNTS : owns
TRADING_ACCOUNTS ||--|| ACCOUNT_SETTINGS : has
TRADING_ACCOUNTS ||--|| ACCOUNT_RISK_LIMITS : has
TRADING_ACCOUNTS ||--o| AI_PROFILES : has
TRADING_ACCOUNTS ||--o{ AI_SESSIONS : runs

INSTRUMENTS ||--o{ MARKET_QUOTES : has
INSTRUMENTS ||--o{ CANDLES : has

TRADING_ACCOUNTS ||--o{ ORDERS : submits
INSTRUMENTS ||--o{ ORDERS : traded_by
ORDERS ||--o{ ORDER_EVENTS : records
ORDERS ||--o{ FILLS : produces

```

```

TRADING_ACCOUNTS ||--o{ POSITIONS : holds
INSTRUMENTS ||--o{ POSITIONS : represents
POSITIONS ||--o{ POSITION_EVENTS : records

AI_STRATEGIES ||--o{ AI_SIGNALS : generates
AI_PROFILES ||--o{ AI_SIGNALS : configures
AI_SIGNALS ||--o{ AI_DECISIONS : evaluated_by
AI_SIGNALS ||--o{ ORDERS : creates
AI_SESSIONS ||--o{ AI_DECISIONS : contains

USER_PROFILES ||--o{ PAYMENT_METHODS : owns
TRADING_ACCOUNTS ||--o{ DEPOSITS : receives
TRADING_ACCOUNTS ||--o{ WITHDRAWALS : pays
PAYMENT_METHODS ||--o{ DEPOSITS : funds
PAYMENT_METHODS ||--o{ WITHDRAWALS : destination

TRADING_ACCOUNTS ||--o{ TRANSACTIONS : records
DEPOSITS ||--o{ TRANSACTIONS : creates
WITHDRAWALS ||--o{ TRANSACTIONS : creates

TRADING_ACCOUNTS ||--o{ LEDGER_ACCOUNTS : owns
LEDGER_JOURNALS ||--o{ LEDGER_ENTRIES : contains
LEDGER_ACCOUNTS ||--o{ LEDGER_ENTRIES : receives
ADMIN_ADJUSTMENTS ||--o{ LEDGER_JOURNALS : posts

TRADING_ACCOUNTS ||--o{ PORTFOLIO_SNAPSHOTS : captures
TRADING_ACCOUNTS ||--o{ DAILY_ACCOUNT_METRICS : summarizes

TRADING_ACCOUNTS ||--o{ PUBLIC_TRADER_PROFILES : exposes
PUBLIC_TRADER_PROFILES ||--o{ LEADERBOARD_SNAPSHOTS : ranks
PUBLIC_TRADER_PROFILES ||--o{ ROBOT_ACTIVITY_EVENTS : publishes
INSTRUMENTS ||--o{ ROBOT_ACTIVITY_EVENTS : references

USER_PROFILES ||--o{ NOTIFICATIONS : receives
TRADING_ACCOUNTS ||--o{ USER_OVERRIDES : receives
TRADING_ACCOUNTS ||--o{ RISK_EVENTS : generates
ORDERS ||--o{ RISK_EVENTS : triggers

```

Standard Fields

Almost har business table mein yeh fields add hon:

```

created_at
updated_at
created_by
updated_by
version
deleted_at, only where soft deletion is allowed

```

Financial, audit, order-event aur ledger tables ko soft delete bhi nahi karna chahiye. Unhein immutable retain kiya jaye.

12. Data Flow Diagram — Context Level

```
flowchart LR
    CLIENT[Client] --> PLATFORM[TradeScope AI Platform]
    ADMIN[Admin] --> PLATFORM

    PLATFORM --> CLIENT
    PLATFORM --> ADMIN

    MARKET[Market Data Provider] --> PLATFORM
    PLATFORM --> BROKER[Broker / Execution Provider]
    BROKER --> PLATFORM

    PLATFORM --> PAYMENT[Payment Provider]
    PAYMENT --> PLATFORM

    NEWS[News Provider] --> PLATFORM
    PLATFORM --> MESSAGE[Email / SMS / Push Provider]
```

13. Data Flow Diagram — Level 1

```
flowchart TB
    CLIENT[Client]
    ADMIN[Admin]

    P1[1. Authentication and User Management]
    P2[2. Market Data Processing]
    P3[3. Order and Trade Processing]
    P4[4. Portfolio and P/L Processing]
    P5[5. AI Signal Processing]
    P6[6. Deposit and Withdrawal Processing]
    P7[7. Live Traders Processing]
    P8[8. Admin and Risk Control]
    P9[9. Notification Processing]

    D1[(Users and Accounts)]
    D2[(Market Data)]
    D3[(Orders, Fills and Positions)]
    D4[(Ledger and Transactions)]
    D5[(AI Signals and Decisions)]
    D6[(Risk and Platform Settings)]
    D7[(Audit Logs)]
```

```
CLIENT --> P1
CLIENT --> P2
CLIENT --> P3
CLIENT --> P4
CLIENT --> P5
CLIENT --> P6
CLIENT --> P7
```

```
ADMIN --> P1
ADMIN --> P6
ADMIN --> P8
```

```
P1 <--> D1
P2 <--> D2
P3 <--> D3
P3 --> D4
P4 <--> D3
P4 <--> D4
P5 <--> D5
P5 --> P3
P5 <--> D6
P6 <--> D4
P7 <--> D3
P8 <--> D6
```

```
P1 --> D7
P3 --> D7
P5 --> D7
P6 --> D7
P8 --> D7
```

```
P3 --> P9
P5 --> P9
P6 --> P9
P8 --> P9
```

```
P9 --> CLIENT
P9 --> ADMIN
```

14. Data Flow Diagram — Order Execution

```
sequenceDiagram
    participant U as Client / AI Engine
    participant API as Trading API
    participant R as Risk Engine
    participant DB as Database
    participant E as Execution Adapter
```

```

participant B as Broker / Simulator
participant L as Ledger Service
participant WS as Realtime Gateway

U->>API: Submit order with idempotency key
API->>DB: Validate account ownership and status
API->>R: Run risk checks

alt Risk rejected
    R-->>API: Rejected with reason codes
    API->>DB: Save order and risk event
    API-->>U: Order rejected
else Risk approved
    R-->>API: Approved
    API->>DB: Save accepted order
    API->>E: Submit order
    E->>B: Place order
    B-->>E: Broker order reference
    E-->>API: Submitted

    B-->>E: Fill or partial fill
    E-->>API: Execution callback
    API->>DB: Save fill and order event
    API->>DB: Create or update position
    API->>L: Post fees and realized effects
    L->>DB: Post balanced ledger journal
    API->>WS: Publish order and position updates
    WS-->>U: Live update
end
end

```

15. Data Flow Diagram — Deposit

```

sequenceDiagram
    participant C as Client
    participant API as Payment API
    participant PSP as Payment Provider
    participant WH as Webhook Handler
    participant DB as Database
    participant L as Ledger Service
    participant WS as Realtime Gateway

    C->>API: Create deposit
    API->>DB: Save deposit intent
    API->>PSP: Create payment session
    PSP-->>API: Payment URL or instructions
    API-->>C: Return payment details

    PSP->>WH: Payment status webhook

```

```

WH->>WH: Verify signature and idempotency
WH->>DB: Store webhook event

alt Payment successful
    WH->>L: Create deposit journal
    L->>DB: Post debit and credit entries
    WH->>DB: Mark deposit completed
    WH->>WS: Publish account balance update
    WS-->>C: Deposit completed
else Payment failed
    WH->>DB: Mark deposit failed
    WH->>WS: Publish failure notification
    WS-->>C: Deposit failed
end

```

16. Data Flow Diagram — Withdrawal

```

sequenceDiagram
    participant C as Client
    participant API as Withdrawal API
    participant R as Finance Risk Checks
    participant DB as Database
    participant A as Finance Admin
    participant PSP as Payout Provider
    participant L as Ledger Service

    C->>API: Submit withdrawal request
    API->>R: Check balance, KYC, margin and limits
    R-->>API: Validation result

    alt Validation failed
        API-->>C: Reject with reason
    else Validation passed
        API->>DB: Save pending withdrawal
        API-->>C: Request pending review
        A->>DB: Review request

        alt Admin rejects
            A->>DB: Mark rejected with reason
        else Admin approves
            A->>PSP: Initiate payout
            PSP-->>A: Payout status

            alt Payout completed
                A->>L: Post withdrawal journal
                L->>DB: Post balanced entries
                A->>DB: Mark completed
            else Payout failed

```

```
        A->>DB: Mark failed or retry
    end
end
end
```

17. Data Flow Diagram — Emergency Stop

```
sequenceDiagram
    participant A as Authorized Admin
    participant API as Admin Control API
    participant DB as Database
    participant AI as AI Engine
    participant T as Trading Service
    participant B as Broker / Simulator
    participant WS as Realtime Gateway

    A->>API: Trigger emergency action with reason
    API->>DB: Create emergency-control record
    API->>AI: Stop new signal execution
    API->>T: Reject new affected orders

    opt Cancel pending orders
        T->>B: Cancel pending orders
        B-->>T: Cancellation results
    end

    opt Close positions
        T->>B: Submit close orders
        B-->>T: Execution results
    end

    API->>DB: Store action results and failures
    API->>WS: Publish platform risk status
    WS-->>A: Emergency action status
```

18. API Structure

Base URL:

```
/api/v1
```

Authentication

```
POST    /auth/register
POST    /auth/login
POST    /auth/logout
POST    /auth/refresh
POST    /auth/forgot-password
POST    /auth/reset-password
POST    /auth/verify-email
GET     /auth/sessions
DELETE  /auth/sessions/{session_id}
```

Client Profile

```
GET     /client/profile
PATCH  /client/profile
GET     /client/kyc
POST    /client/kyc
POST    /client/kyc/documents
```

Trading Accounts

```
GET     /client/accounts
POST    /client/accounts
GET     /client/accounts/{account_id}
PATCH  /client/accounts/{account_id}
GET     /client/accounts/{account_id}/metrics
```

Dashboard

```
GET /client/accounts/{account_id}/dashboard
GET /client/accounts/{account_id}/portfolio-history
GET /client/accounts/{account_id}/daily-metrics
GET /client/accounts/{account_id}/risk-summary
```

Market Data

```
GET /market/instruments
GET /market/instruments/{symbol}
GET /market/quotes
GET /market/candles
GET /market/status
```

Orders

```
POST /client/accounts/{account_id}/orders
GET /client/accounts/{account_id}/orders
GET /client/accounts/{account_id}/orders/{order_id}
POST /client/accounts/{account_id}/orders/{order_id}/cancel
PATCH /client/accounts/{account_id}/orders/{order_id}
```

Positions

```
GET /client/accounts/{account_id}/positions
GET /client/accounts/{account_id}/positions/{position_id}
POST /client/accounts/{account_id}/positions/{position_id}/close
POST /client/accounts/{account_id}/positions/{position_id}/partial-close
PATCH /client/accounts/{account_id}/positions/{position_id}/protection
```

AI

```
GET /client/accounts/{account_id}/ai/settings
PATCH /client/accounts/{account_id}/ai/settings
POST /client/accounts/{account_id}/ai/start
POST /client/accounts/{account_id}/ai/pause
GET /client/accounts/{account_id}/ai/signals
POST /client/accounts/{account_id}/ai/signals/{signal_id}/approve
POST /client/accounts/{account_id}/ai/signals/{signal_id}/reject
```

Deposits and Withdrawals

```
GET /client/payment-methods
POST /client/payment-methods
POST /client/accounts/{account_id}/deposits
GET /client/accounts/{account_id}/deposits
POST /client/accounts/{account_id}/withdrawals
GET /client/accounts/{account_id}/withdrawals
GET /client/accounts/{account_id}/transactions
```

Live Traders

```
GET /live-traders/leaderboard
GET /live-traders/activity
PATCH /client/accounts/{account_id}/leaderboard-preferences
```

News

```
GET /news
GET /news/{news_id}
```

Admin Users

```
GET /admin/users
POST /admin/users
GET /admin/users/{user_id}
PATCH /admin/users/{user_id}
POST /admin/users/{user_id}/activate
POST /admin/users/{user_id}/suspend
POST /admin/users/{user_id}/restrict
POST /admin/users/{user_id}/ai/enable
POST /admin/users/{user_id}/ai/disable
```

Admin Finance

```
GET /admin/finance/overview
GET /admin/transactions
GET /admin/deposits
GET /admin/withdrawals
POST /admin/withdrawals/{id}/approve
POST /admin/withdrawals/{id}/reject
POST /admin/accounts/{account_id}/adjustments
GET /admin/ledger/journals
```

Admin AI and Risk

```
GET /admin/ai/overview
GET /admin/ai/sessions
PATCH /admin/ai/global-settings
PATCH /admin/ai/users/{account_id}/override
POST /admin/risk/emergency-stop
POST /admin/risk/emergency-clear
GET /admin/risk/events
GET /admin/risk/exposure
```

Platform Settings

```
GET    /admin/platform/settings
PATCH /admin/platform/settings
GET    /admin/platform/settings/history
```

Audit and Reports

```
GET    /admin/audit-logs
GET    /admin/reports
POST   /admin/reports/export
GET    /admin/reports/exports/{export_id}
```

19. Realtime WebSocket Events

WebSocket endpoint:

```
/ws/v1
```

Market Events

```
market.tick
market.quote.updated
market.candle.updated
market.status.changed
```

Trading Events

```
order.created
order.accepted
order.submitted
order.partially_filled
order.filled
order.rejected
order.canceled
position.opened
position.updated
position.closed
```

Account Events

```
account.balance.updated  
account.metrics.updated  
account.margin.warning  
account.daily_limit.updated
```

AI Events

```
ai.session.started  
ai.session.paused  
ai.signal.created  
ai.signal.expired  
ai.order.proposed  
ai.order.rejected_by_risk
```

Finance Events

```
deposit.updated  
withdrawal.updated  
transaction.created
```

Live Traders Events

```
leaderboard.updated  
robot_activity.created  
trader_rank.changed
```

Admin Events

```
user.status.changed  
risk.event.created  
platform.emergency_stop  
platform.settings.updated
```

Every event mein yeh fields hon:

```
{  
  "event_id": "uuid",  
  "event_type": "position.updated",  
  "account_id": "uuid",  
  "sequence": 17281,  
  "occurred_at": "timestamp",
```

```
"data": {}  
}
```

`sequence` frontend ko missed ya out-of-order events detect karne mein help karega.

20. Idempotency and Concurrency Controls

Financial aur trading requests idempotent hon:

```
POST /orders  
POST /deposits  
POST /withdrawals  
POST /admin/adjustments
```

Client header:

```
Idempotency-Key: unique-value
```

Backend:

- Request hash store kare.
- Same key aur same payload par previous response return kare.
- Same key aur different payload reject kare.
- Database transaction use kare.
- Account-level critical operations par lock use kare.
- Duplicate webhooks external event ID se block kare.

21. Development Milestones

Client Side — 10 Milestones

Milestone	Scope	Completion Result
C1	Authentication, profile, protected routes, client layout	Client securely Client Zone access kar sake
C2	Multiple trading accounts, account switcher, account status	Client accounts create aur switch kar sake
C3	Instruments, quotes, historical candles, market feed	Market prices aur candle data available ho
C4	Professional Web Trader chart, zoom, pan, live candle updates	TradingView-style chart experience work kare

Milestone	Scope	Completion Result
C5	Order ticket, order validation, simulation execution, positions	Client manual trades execute aur manage kar sake
C6	Portfolio dashboard, balance, equity, margin, live P/L	Complete financial dashboard live update ho
C7	AI profiles, AI signals, confidence, order board, risk checks	AI recommendations aur controlled auto-trading work kare
C8	Live Traders leaderboard and robot activity feed	Anonymous leaderboard real time update ho
C9	Deposits, withdrawals, payment methods, transaction history	Complete account funding flow work kare
C10	Notifications, KYC, security, responsive testing and Client UAT	Client Zone production-ready ho

Admin Side — 10 Milestones

Milestone	Scope	Completion Result
A1	Admin authentication, 2FA, RBAC, admin layout	Secure admin access ready ho
A2	User directory, user detail, status management	Complete user administration work kare
A3	Trading account administration and account overrides	Admin individual accounts control kar sake
A4	Double-entry ledger, balances and admin adjustments	Financial source of truth complete ho
A5	Deposit and withdrawal oversight, approval workflow	Finance operations manageable hon
A6	Transaction monitoring, filters, details and exports	Platform-wide transaction oversight available ho
A7	Global AI settings, user AI overrides and monitoring	Admin AI engine control kar sake
A8	Risk exposure dashboard, risk events and emergency controls	Risk manager platform halt/control kar sake
A9	Platform settings, fees, leverage and configurable limits	Business configuration admin se manage ho
A10	Audit logs, reports, observability, security hardening and Admin UAT	Admin Console and platform production-ready hon

22. Recommended Implementation Order

```
graph TD; A[Foundation and Security] --> B[User and Trading Accounts]; B --> C[Market Data]; C --> D[Web Trader Chart]; D --> E[Orders and Simulation Execution]; E --> F[Positions and Portfolio Calculations]; F --> G[Double-Entry Ledger]; G --> H[Deposits and Withdrawals]; H --> I[AI Signal Engine]; I --> J[Risk Engine]; J --> K[Live Traders]; K --> L[Admin Controls]; L --> M[Production Hardening]; M --> N[Broker and Payment Integrations];
```

Foundation and Security
↓
User and Trading Accounts
↓
Market Data
↓
Web Trader Chart
↓
Orders and Simulation Execution
↓
Positions and Portfolio Calculations
↓
Double-Entry Ledger
↓
Deposits and Withdrawals
↓
AI Signal Engine
↓
Risk Engine
↓
Live Traders
↓
Admin Controls
↓
Production Hardening
↓
Broker and Payment Integrations

AI trading ko order engine aur risk engine se pehle build nahi karna chahiye.

Live-money integrations ko double-entry ledger, audit trail aur idempotency complete hone se pehle enable nahi karna chahiye.

23. Testing Strategy

Unit Tests

- P/L calculations
- Margin calculations
- Risk limits
- Position aggregation
- Fee calculations
- Ledger balancing
- AI limit hierarchy

- Order state transitions

Integration Tests

- Market tick to candle update
- Order to fill to position
- Fill to P/L update
- Deposit webhook to ledger
- Withdrawal approval to payout
- AI signal to order
- Emergency stop to order rejection

End-to-End Tests

Client:

- Registration
- Account creation
- Deposit
- Place trade
- Modify trade
- Close trade
- AI settings
- Withdrawal

Admin:

- User suspension
- Balance adjustment
- Withdrawal approval
- AI override
- Emergency stop
- CSV export

Load Tests

- Concurrent market-data connections
- WebSocket subscriptions
- Live P/L updates
- Leaderboard updates
- Order bursts
- Payment webhooks

Financial Integrity Tests

Every test ke baad:

Total debits = Total credits

Check ho.

Failure Tests

- Market provider disconnected
 - Broker timeout
 - Duplicate webhook
 - Duplicate order request
 - Payment callback out of order
 - Database temporary failure
 - WebSocket reconnect
 - Emergency stop during order submission
-

24. Security Requirements

Authentication

- JWT validation
- Refresh-token rotation
- Admin 2FA
- Session revocation
- Secure password-reset flow

Authorization

- Supabase RLS
- Backend ownership validation
- Admin RBAC
- Field-level restrictions
- No client-provided balance or role trust

API Protection

- Rate limiting
- CORS allowlist
- Request-size limits
- Input validation
- SQL-injection prevention
- Idempotency
- Signed webhooks
- Replay-attack protection

Data Protection

- TLS everywhere
- Encrypted secrets
- Sensitive provider tokens encrypted
- Payment data tokenized

- KYC documents private storage buckets
- Signed temporary storage URLs
- Audit access to sensitive data

Financial Security

- Immutable ledger
- Maker-checker approvals
- Withdrawal velocity limits
- Destination verification
- Large transaction review
- Suspicious behavior flags

Operational Security

- Secret rotation
 - Separate development/staging/production secrets
 - Restricted production database access
 - Automated backups
 - Point-in-time recovery
 - Error tracking
 - Security alerts
-

25. Audit Logging

Audit events:

- Login and logout
- Failed admin login
- User status changes
- Role changes
- KYC decisions
- AI settings changes
- Risk-limit changes
- Balance adjustments
- Withdrawal decisions
- Emergency actions
- Platform-settings changes
- Sensitive data access

Audit log should contain:

```
actor
role
action
resource
previous value
new value
```

```
reason
IP address
user agent
timestamp
correlation ID
```

Audit logs clients aur normal admins ke through edit/delete nahi hon.

26. Observability

Logs

Structured JSON logs:

- Request ID
- User ID
- Account ID
- Service
- Event
- Duration
- Status
- Error code

Metrics

- API latency
- API error rate
- Active WebSockets
- Market-feed delay
- Order execution latency
- Rejected orders
- Broker failures
- AI signal count
- Risk rejections
- Ledger posting failures
- Payment webhook failures

Alerts

Immediate alerts:

- Market feed stale
- Broker unavailable
- Ledger imbalance
- Payment webhook failures
- Abnormal withdrawal volume
- AI error spike
- Database connection failure

- Emergency stop triggered
-

27. Deployment Environments

Development

- Local frontend
- Local FastAPI
- Development Supabase project
- Simulated market/execution provider

Staging

- Cloudflare Pages staging
- Render staging backend
- Staging Supabase
- Payment sandbox
- Broker sandbox
- Realistic test accounts

Production

- Cloudflare Pages production
- Render production service
- Production Supabase
- Redis/queue
- Production market provider
- Production broker/payment providers
- Monitoring and alerting

Development, staging and production databases completely separate.

28. CI/CD Pipeline

On every pull request:

1. TypeScript lint
2. Python lint
3. Type checking
4. Unit tests
5. Integration tests
6. Database migration validation
7. Security/dependency scan
8. Frontend build
9. Backend build

Production deployment:

1. Approved pull request
2. Staging deployment
3. Smoke tests
4. Migration dry run
5. Production backup
6. Additive database migration
7. Backend deployment
8. Frontend deployment
9. Post-deployment checks

Database migrations additive aur idempotent hon.

29. Non-Functional Requirements

Performance

- Standard API response target under normal load: under 500 ms
- Trading commands ko minimum possible latency
- WebSocket market updates batched where required
- Dashboard initial load optimized
- Large tables server-side paginated

Reliability

- No duplicate financial transaction
- No duplicate order from retries
- Reconnectable WebSockets
- Provider outage handling
- Background-job retries with limits
- Dead-letter queue for failed events

Scalability

- Stateless API instances
- Shared Redis
- Event-driven real-time updates
- Indexed database queries
- Time-series market data partitioning
- Archived historical candles

Accessibility

- Keyboard navigation
- High contrast
- Clear profit/loss indicators
- Profit/loss ko sirf color se represent na kiya jaye

- Responsive desktop, tablet and mobile layouts
-

30. Production Launch Gates

Real-money launch se pehle following gates pass hon:

- Client authorization fully enforced
 - Supabase RLS reviewed
 - Rate limits active
 - CORS restricted
 - Secrets rotated
 - Admin 2FA active
 - Double-entry ledger verified
 - Deposit webhook signatures verified
 - Withdrawal approval workflow complete
 - Idempotency tests passed
 - Broker reconciliation complete
 - Market-feed failover defined
 - Emergency stop tested
 - Backups and recovery tested
 - Audit logs immutable
 - KYC/AML workflow integrated
 - Applicable legal and regulatory approvals confirmed
-

31. Definition of Done

Feature tab complete samjha jayega jab:

- Frontend implemented ho
 - Backend API implemented ho
 - Database migration available ho
 - Authorization applied ho
 - Validation complete ho
 - Audit logs added hon
 - Error states designed hon
 - Loading and empty states hon
 - Mobile responsive ho
 - Unit tests pass hon
 - Integration tests pass hon
 - Documentation updated ho
 - Staging environment par acceptance criteria pass hon
-

32. Final System Flow

```
flowchart LR
    USER[Client] --> ACCOUNT[Trading Account]
    ACCOUNT --> SETTINGS[AI and Risk Settings]
    MARKET[Live Market Data] --> ENGINE[Market and AI Engine]
    SETTINGS --> ENGINE
    ENGINE --> SIGNAL[AI Signal]
    SIGNAL --> RISK[Risk Validation]
    USER --> ORDER[Manual Order]
    ORDER --> RISK
    RISK --> EXECUTION[Broker or Simulation Execution]
    EXECUTION --> FILL[Trade Fill]
    FILL --> POSITION[Position]
    POSITION --> PNL[Live P/L]
    PNL --> DASHBOARD[Client Dashboard]
    FILL --> LEDGER[Double-Entry Ledger]
    LEDGER --> BALANCE[Balance and Equity]
    BALANCE --> DASHBOARD
    POSITION --> LIVE[Anonymous Live Traders]
    ADMIN[Admin Console] --> ACCOUNT
    ADMIN --> SETTINGS
    ADMIN --> RISK
    ADMIN --> LEDGER
```

TradeScope AI ka central principle yeh hona chahiye:

```
Market Data drives prices
Orders drive fills
Fills drive positions
Positions drive P/L
Ledger drives financial balances
Risk Engine controls every trade
Audit Logs record every sensitive action
```